



☕ Spring Framework: Advanced Bean Management - Day 10



JAVA 21



SPRING 7.0.3



MAVEN



BEAN MANAGEMENT



© 2026 Avinash Dhanuka

Advanced Spring Bean Management Mastery

From Lifecycle to Performance - The Complete Journey



GITHUB

AVINASH-706



CONTACT ME VIA GMAIL

Learning Path: This repository contains three comprehensive projects demonstrating advanced Spring Framework bean management - from life-cycle management to performance optimization with lazy loading, including real-world case studies.

Table of Contents

1. [Overview](#)
 2. [Projects Summary](#)
 3. [Project 1: Bean Lifecycle Management](#)
 4. [Project 2: Lazy Configuration](#)
 5. [Project 3: Real-World Case Studies](#)
 6. [Key Concepts Learned](#)
 7. [Configuration Comparison](#)
 8. [When to Use What](#)
 9. [Quick Reference](#)
 10. [Running the Projects](#)
-

OVERVIEW



This repository demonstrates advanced Spring Framework bean management techniques, focusing on lifecycle management, performance optimization, and real-world application patterns. Each project builds upon core Spring concepts to solve practical problems.

Learning Objectives

- ✓ Master Bean Lifecycle with `@PostConstruct` and `@PreDestroy`
- ✓ Optimize performance with `@Lazy` initialization
- ✓ Understand Bean Scopes (Singleton vs Prototype)
- ✓ Resolve dependency ambiguity with `@Primary` and `@Qualifier`
- ✓ Implement proper resource management
- ✓ Apply patterns in real-world scenarios

Repository Structure

day10/

```

└─ BeanLifecycle/                                     # Project 1: Lifecycle management
    └─ src/
        └─ main/
            └─ java/
                └─ org/example/
                    └─ lifecycle/
                        ├── dbConnection.java
                        ├── LifeCycleConfig.java
                        └── LifeCycleDemo.java
    └─ pom.xml
    └─ README.md                                     # Detailed documentation

└─ LazyConfig/                                       # Project 2: Lazy initialization
    └─ src/
        └─ main/
            └─ java/
                └─ org/example/
                    └─ lazy/
                        ├── EagerBean.java
                        ├── LazyBean.java
                        ├── LazyConfig.java
                        └── LazyDemo.java
    └─ pom.xml
    └─ README.md                                     # Detailed documentation

└─ CaseStudy/                                       # Project 3: Real-world applications
    └─ BankLoanApproval/
        └─ src/main/java/org/example/bankloan/
    └─ FoodDelivery/
        └─ src/main/java/org/example/fooddelivery/
    └─ PaymentProcessing/
        └─ src/main/java/org/example/payment/
    └─ pom.xml
    └─ README.md                                     # Detailed documentation

└─ README.md                                         # This file

```

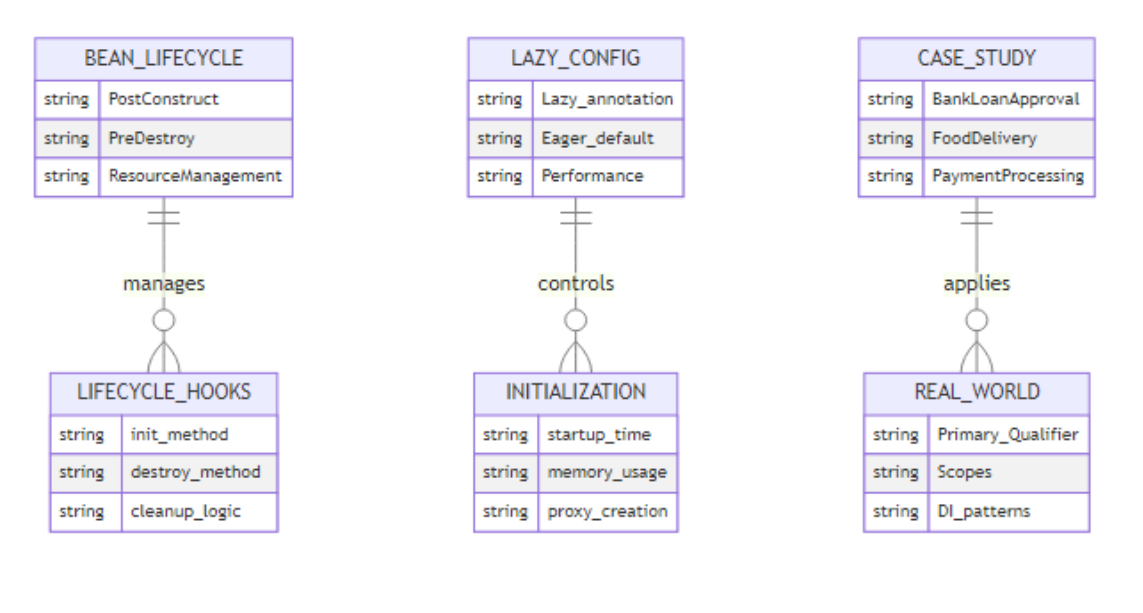
PROJECTS SUMMARY



 Project Overview Table

Project	Focus Area	Key Concepts	Complexity	Status
BeanLifeCycle	Resource Management	@PostConstruct, @PreDestroy, Lifecycle	☆☆☆ Intermediate	✓ Complete
LazyConfig	Performance	@Lazy, Eager vs Lazy, Proxies	☆☆☆ Intermediate	✓ Complete
CaseStudy	Real-World Apps	@Primary, @Qualifier, Scopes, All concepts	☆☆☆☆ Advanced	✓ Complete

 Project Relationships



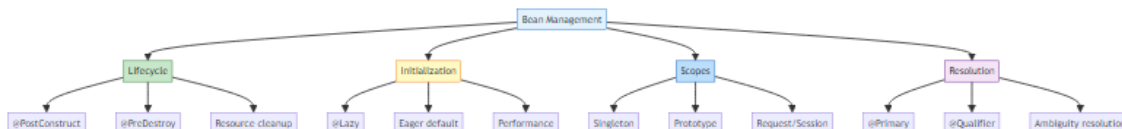
BEAN LIFECYCLE EVOLUTION



Spring Bean Management Timeline



Bean Management Approaches



PROJECT 1: BEAN LIFECYCLE MANAGEMENT



📁 **Resource Management by:** Avinash Dhanuka | © 2026

🚀 Overview

Understanding and managing the complete lifecycle of Spring beans from creation to destruction. This project demonstrates proper resource management using `@PostConstruct` and `@PreDestroy` annotations.

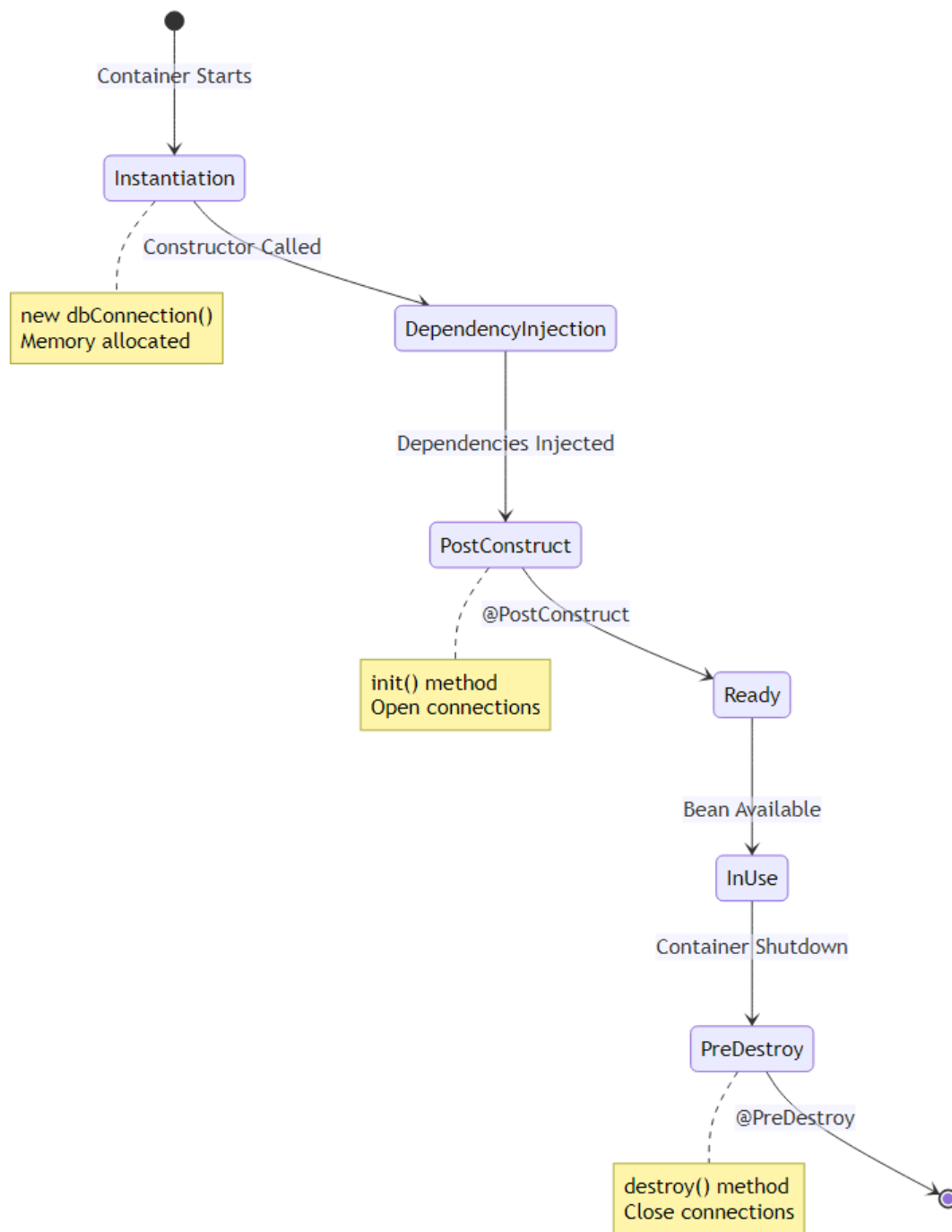
📁 **Location:** [BeanLifeCycle/](#)

📖 **Full Documentation:** [BeanLifeCycle/README.md](#)

🔑 Key Concepts

- **Bean Lifecycle Phases** (13 phases from instantiation to destruction)
- **@PostConstruct** for initialization after dependency injection
- **@PreDestroy** for cleanup before bean destruction
- **Resource Management** (database connections, file handles)
- **Lifecycle Callbacks** (InitializingBean, DisposableBean)
- **BeanPostProcessor** for custom processing

Bean Lifecycle Flow



🔗 Implementation Example

Database Connection Bean:

```
@Component
public class dbConnection {
    // Phase 1: Constructor
    public dbConnection() {
        System.out.println("DB Constructor is called !!");
    }

    // Phase 7: Initialization
    @PostConstruct
    public void init() {
        System.out.println("Init Method is called !!");
        // Open database connection
        // Load configuration
    }

    // Business method
    public void executeQuery() {
        System.out.println("Query is being Executed !!");
    }

    // Phase 12: Cleanup
    @PreDestroy
    public void destroy() {
        System.out.println("Destroy method called");
        // Close database connection
        // Release resources
    }
}
```

🔍 What You'll Learn

1. Complete Lifecycle Phases

- Instantiation → DI → Awareness → Initialization → Ready → Destruction
- 13 distinct phases in bean lifecycle
- When each phase executes

2. @PostConstruct Usage

- Called after dependency injection
- Guaranteed dependencies are available
- Perfect for initialization logic

- JSR-250 standard annotation
- 3. **@PreDestroy Usage**
 - Called before bean destruction
 - Cleanup resources properly
 - Prevent memory leaks
 - Graceful shutdown
- 4. **Resource Management**
 - Opening database connections
 - Closing file handles
 - Releasing network resources
 - Flushing buffers
- 5. **Lifecycle Alternatives**
 - InitializingBean interface
 - DisposableBean interface
 - Custom init/destroy methods
 - Comparison and best practices

Lifecycle Methods Comparison

Method	Pros	Cons	Use Case
@PostConstruct	Standard, no coupling	Requires annotation support	✓ Recommended
InitializingBean	Type-safe, IDE support	Spring coupling	Legacy code
init-method (XML)	Works with any class	Only XML config	Third-party beans
@PreDestroy	Standard, no coupling	Requires annotation support	✓ Recommended
DisposableBean	Type-safe	Spring coupling	Legacy code
destroy-method (XML)	Works with any class	Only XML config	Third-party beans

🎓 Key Takeaways

- @PostConstruct runs AFTER dependency injection
 - @PreDestroy runs BEFORE bean destruction
 - Use for proper resource management
 - Prevents memory leaks and resource exhaustion
 - Standard JSR-250 annotations (no Spring coupling)
-

PROJECT 2: LAZY CONFIGURATION



📌 **Performance Optimization by:** Avinash Dhanuka | © 2026

🚀 Overview

Performance optimization through lazy bean initialization. This project demonstrates how @Lazy annotation delays bean creation until first use, improving startup time and memory efficiency.

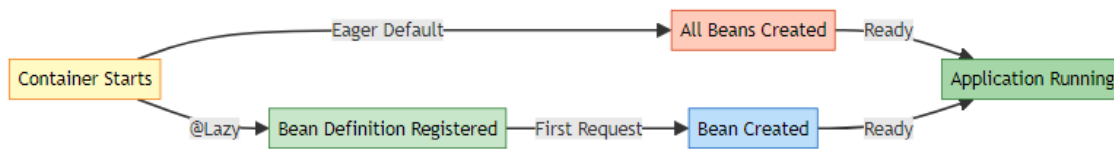
📁 **Location:** [LazyConfig/](#)

📖 **Full Documentation:** [LazyConfig/README.md](#)

🎯 Key Concepts

- **@Lazy Annotation** for delayed initialization
- **Eager vs Lazy** initialization comparison
- **Proxy Mechanism** (CGLIB and JDK proxies)
- **Performance Impact** on startup and memory
- **Lazy with DI** (constructor, setter, field injection)
- **Breaking Circular Dependencies**

📺 Eager vs Lazy Initialization



🔧 Implementation Example

Eager Bean (Default):

```
@Component
public class EagerBean {
    public EagerBean() {
        System.out.println("Eager Bean Created !!");
        // Created at container startup
    }
}
```

Lazy Bean:

```
@Component
@Lazy
public class LazyBean {
    public LazyBean() {
        System.out.println("Lazy Bean Created !!");
        // Created on first access
    }
}
```

Execution:

```
ApplicationContext context = new
    AnnotationConfigApplicationContext(LazyConfig.class);
// Output: "Eager Bean Created !!"
// NO output for LazyBean yet!
```

```
LazyBean bean = context.getBean(LazyBean.class);
// NOW output: "Lazy Bean Created !!"
```

🔍 What You'll Learn

1. The Problem with Eager Initialization

- Slow startup time
 - Memory waste for unused beans
 - Resource lock (connections opened early)
 - Unnecessary initialization work
2. **How @Lazy Works**
 - Bean definition registered only
 - Proxy created for injection
 - Real bean created on first method call
 - Internal proxy mechanism
 3. **@Lazy at Different Levels**
 - Class level: @Component @Lazy
 - Method level: @Bean @Lazy
 - Parameter level: @Autowired @Lazy
 - Configuration level: @Configuration @Lazy
 4. **Proxy Mechanism**
 - JDK Dynamic Proxy (interface-based)
 - CGLIB Proxy (class-based)
 - Proxy intercepts method calls
 - First call triggers bean creation
 5. **Performance Impact**
 - Faster startup time
 - Lower memory at startup
 - Slight overhead on first access
 - Trade-offs and considerations

Performance Comparison

Metric	Eager (100 beans)	Lazy (50% used)
Startup Time	30 seconds	5 seconds
Memory at Startup	500 MB	250 MB
First Request	10 ms	15 ms (proxy)
Subsequent Requests	10 ms	10 ms
Total Memory	500 MB	400 MB

🔗 When to Use @Lazy

Use Lazy When: - ✓ Bean is rarely used - ✓ Initialization is expensive - ✓ Want fast startup - ✓ Optional features

Use Eager When: - ✓ Bean used in every request - ✓ Want fail-fast behavior - ✓ Initialization is quick - ✓ Need predictable performance

📌 Key Takeaways

- @Lazy delays bean creation until first use
 - Improves startup time and memory usage
 - Creates proxy for dependency injection
 - Trade-off: first access is slower
 - Perfect for rarely-used expensive beans
-

PROJECT 3: REAL-WORLD CASE STUDIES



📌 **Practical Applications by:** Avinash Dhanuka | © 2026

✈️ Overview

Three production-ready Spring applications demonstrating how all concepts work together in real-world scenarios. Each case study solves a specific business problem using advanced Spring patterns.

📁 **Location:** [CaseStudy/](#)

📖 **Full Documentation:** [CaseStudy/README.md](#)

🔗 Case Studies

1️⃣ Bank Loan Approval System

Business Problem: Validate loan applications using different strategies

Key Features: - @Qualifier to select IncomeValidator over CreditScoreValidator - Prototype scope for fresh validator instances - @Lazy AuditService for optional auditing - Setter injection for optional dependencies

Technologies:

```
@Component
public class LoanService {
    @Autowired
    public LoanService(@Qualifier("incomeValidator") LoanValidator
        validator) {
        // Explicitly select income-based validation
    }
}
```

2️⃣ Food Delivery System

Business Problem: Multi-channel notification system for orders

Key Features: - @Primary EmailNotification as default - @Qualifier for SMS in urgent orders - @Lazy SMS gateway (expensive connection) - Singleton Delivery-Service with lifecycle management

Technologies:

```
@Component
public class OrderService {
    public OrderService(@Qualifier("smsNotification") NotificationService
        service) {
        // Override @Primary for urgent notifications
    }
}
```

3️⃣ Payment Processing System

Business Problem: Support multiple payment methods

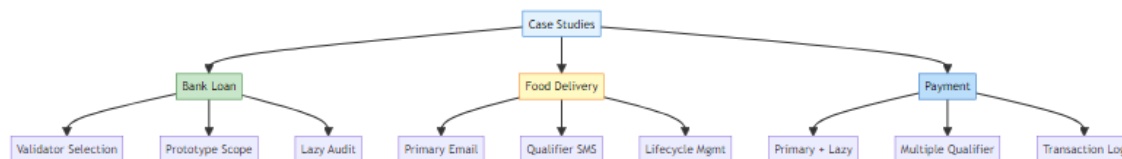
Key Features: - @Primary + @Lazy for CreditCardPayment - Prototype UpiPayment for transaction isolation - Multiple @Qualifier in single service - TransactionLogger with lifecycle hooks

Technologies:

@Component

```
public class PaymentProcessor {  
    public PaymentProcessor(  
        @Qualifier("upiPayment") PaymentService upi,  
        @Qualifier("creditCardPayment") PaymentService card) {  
        // Use both payment methods  
    }  
}
```

Case Studies Architecture



What You'll Learn

- 1. Dependency Resolution Patterns**
 - When to use @Primary vs @Qualifier
 - Combining @Primary with @Lazy
 - Multiple @Qualifier in one service
 - Resolution priority rules
- 2. Bean Scope Selection**
 - Singleton for shared services
 - Prototype for stateful objects
 - Scope impact on performance
 - Real-world use cases
- 3. Performance Optimization**
 - Lazy loading expensive resources
 - Balancing startup vs runtime
 - Memory management strategies
 - Resource pooling patterns
- 4. Lifecycle Management**

- Opening connections in @PostConstruct
- Closing resources in @PreDestroy
- Graceful shutdown patterns
- Error handling in lifecycle

5. Dependency Injection Patterns

- Constructor for required dependencies
- Setter for optional dependencies
- Field injection considerations
- Best practices

📊 Concepts Integration Matrix

Concept	Bank Loan	Food Delivery	Payment
@Primary	✓ CreditScore	✓ Email	✓ CreditCard
@Qualifier	✓ Income	✓ SMS	✓ UPI + Card
@Lazy	✓ Audit	✓ SMS	✓ CreditCard
Singleton	✓ LoanService	✓ DeliveryService	✓ Logger
Prototype	✓ Validator	✗	✓ UpiPayment
Lifecycle	✓ Audit	✓ Delivery	✓ Logger

📌 Key Takeaways

- Real-world apps combine multiple Spring concepts
- @Primary for defaults, @Qualifier for specific needs
- Lazy loading improves startup performance
- Proper lifecycle management prevents resource leaks
- Choose scope based on statefulness

KEY CONCEPTS LEARNED



🎯 Day 10 Complete Learning Mindmap



🎯 Core Spring Concepts

1. Bean Lifecycle Management

Why It Matters: - Proper resource initialization - Guaranteed cleanup - Memory leak prevention - Graceful shutdown

Key Phases:



Best Practices: - Use @PostConstruct for initialization - Use @PreDestroy for cleanup - Don't use constructor for DI-dependent logic - Handle exceptions in lifecycle methods

2. Lazy Initialization

Why It Matters: - Faster application startup - Lower memory footprint - On-demand resource allocation - Better user experience

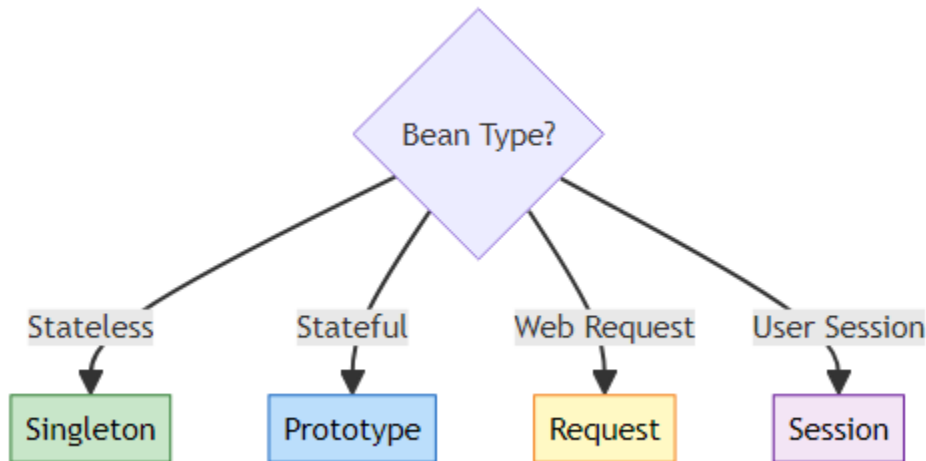
Comparison: | Aspect | Eager | Lazy | | Startup | Slow | Fast | | Memory | High | Low | | First Access | Fast | Slow | | Error Detection | Early | Late |

Best Practices: - Use @Lazy for expensive beans - Use @Lazy for rarely-used features - Combine with @Primary for defaults - Consider first-access delay

3. Bean Scopes

Why It Matters: - Controls instance creation - Manages state and memory - Affects performance - Determines lifecycle

Scope Selection Guide:

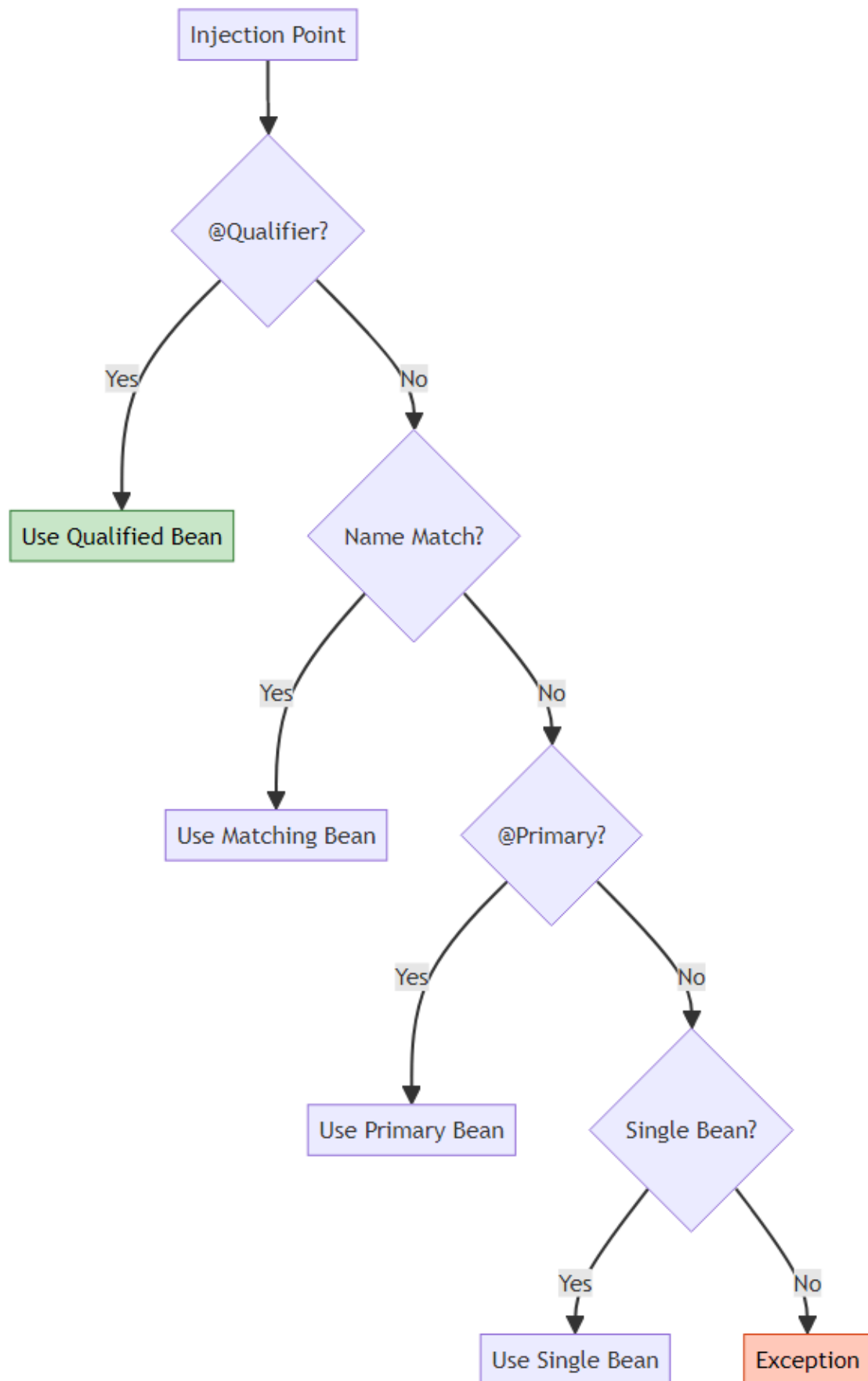


Best Practices: - Default to Singleton for services - Use Prototype for stateful objects - Avoid Prototype for expensive beans - Consider memory implications

4. Dependency Resolution

Why It Matters: - Resolves ambiguity - Provides flexibility - Enables polymorphism - Supports multiple implementations

Resolution Flow:

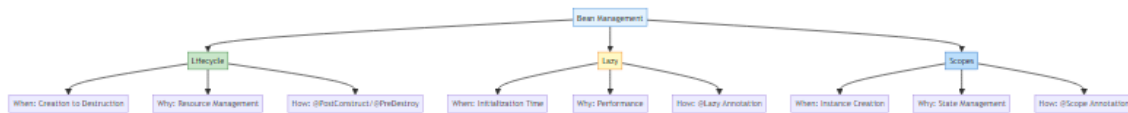


Best Practices: - Use @Primary for default implementation - Use @Qualifier for specific selection - Only ONE @Primary per type - Clear bean naming conventions

LIFECYCLE VS LAZY VS SCOPES



📊 Detailed Comparison



📋 Feature Comparison Table

Feature	Lifecycle	Lazy	Scopes
Purpose	Resource management	Performance optimization	Instance control
Annotations	@PostConstruct, @PreDestroy	@Lazy	@Scope
When Applied	Bean creation/destruction	Bean initialization	Bean instantiation
Affects	Initialization/cleanup	Creation timing	Instance count
Default	No lifecycle hooks	Eager initialization	Singleton
Use Case	DB connections, cleanup	Expensive beans	Stateful objects
Performance	Minimal overhead	Faster start-up	Varies by scope

Feature	Lifecycle	Lazy	Scopes
Memory	No direct impact	Lower at startup	Varies by scope
Complexity	Low	Medium	Medium

🔗 When to Use Each

Use Lifecycle Management When:

- ✓ Need to open database connections
- ✓ Need to load configuration files
- ✓ Need to close resources properly
- ✓ Need graceful shutdown
- ✓ Need to prevent memory leaks

Example:

```

@Component
public class DatabaseService {
    private Connection connection;

    @PostConstruct
    public void init() {
        connection = openConnection();
    }

    @PreDestroy
    public void cleanup() {
        connection.close();
    }
}

```

Use Lazy Initialization When:

- ✓ Bean is rarely used
- ✓ Initialization is expensive (5+ seconds)
- ✓ Want fast application startup
- ✓ Optional features that may not be used
- ✓ Breaking circular dependencies

Example:

```
@Component
@Lazy
public class ReportGenerator {
    public ReportGenerator() {
        // Expensive: Load templates, initialize PDF Library
    }
}
```

Use Appropriate Scope When:

- ✓ Singleton: Stateless services (default)
- ✓ Prototype: Stateful objects, per-request data
- ✓ Request: Web request-specific data
- ✓ Session: User session data

Example:

```
@Component
@Scope("prototype")
public class ShoppingCart {
    private List<Item> items = new ArrayList<>();
    // Each user gets their own cart
}
```

🔄 Combining Concepts

Scenario 1: Lazy + Lifecycle

```
@Component
@Lazy
public class EmailService {
    @PostConstruct
    public void init() {
        // Connect to SMTP server (only when first used)
    }

    @PreDestroy
    public void cleanup() {
        // Close SMTP connection
    }
}
```

Scenario 2: Prototype + Lifecycle

```
@Component
@Scope("prototype")
public class FileProcessor {
    @PostConstruct
    public void init() {
        // Open file handle
    }

    @PreDestroy
    public void cleanup() {
        // ⚠ WARNING: @PreDestroy NOT called for Prototype!
        // Must manually manage cleanup
    }
}
```

Scenario 3: Lazy + Scope + Lifecycle

```
@Component
@Lazy
@Scope("singleton")
public class CacheManager {
    @PostConstruct
    public void init() {
        // Initialize cache (lazy, singleton)
    }

    @PreDestroy
    public void cleanup() {
        // Flush cache to disk
    }
}
```

⚠ Important Interactions

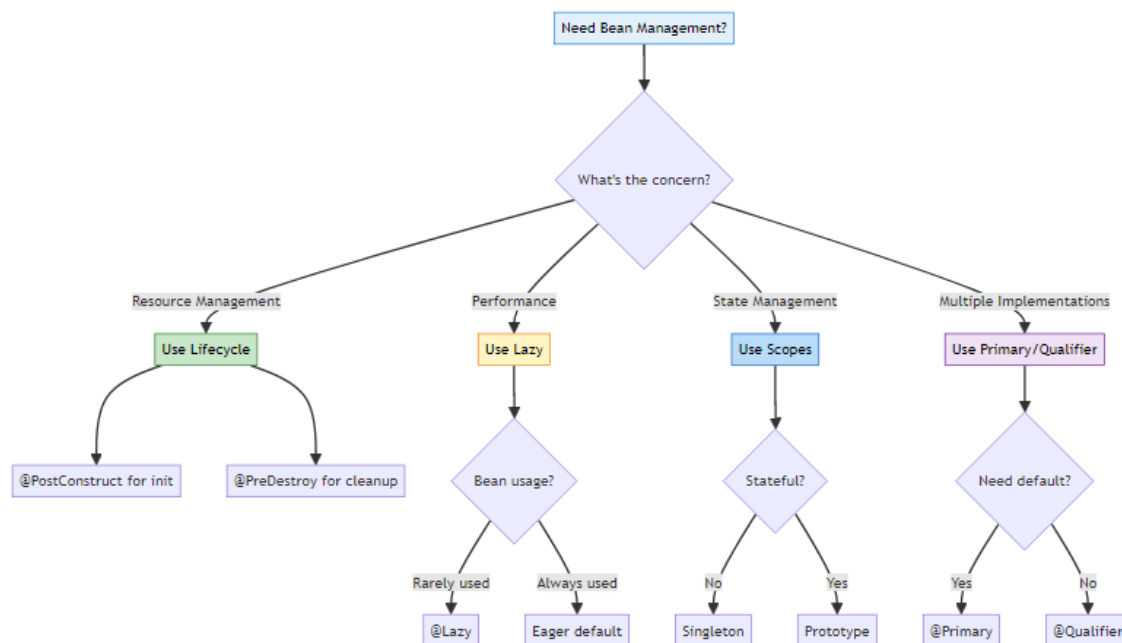
Combination	Behavior	Notes
Lazy + Singleton	✓ Works perfectly	Bean created once, on first use
Lazy + Prototype	⚠ Partial	Each request creates new instance

Combination	Behavior	Notes
Prototype + @PreDestroy	✗ Not called	Spring doesn't manage proto-type destruction
Lazy + @PostConstruct	✓ Works	@PostConstruct called when bean created
Singleton + @PreDestroy	✓ Works	Called on container shutdown

WHEN TO USE WHAT



🎯 Decision Tree



Use Case Matrix

Scenario	Solution	Example
Database Connection	Lifecycle	@PostConstruct open, @PreDestroy close
Expensive Report Generator	Lazy	@Lazy for 5-second initialization
User Shopping Cart	Prototype	@Scope("prototype") for per-user state
Email Service (default)	Primary	@Primary for default notification
SMS Service (specific)	Qualifier	@Qualifier("sms") for urgent alerts
Cache Manager	Lazy + Lifecycle	@Lazy @PostConstruct @PreDestroy
Payment Gateway	Primary + Lazy	@Primary @Lazy for default but expensive
Transaction Logger	Singleton + Lifecycle	Default scope with lifecycle hooks

Real-World Scenarios

Scenario 1: E-Commerce Application

Requirements: - Fast startup - Multiple payment methods - User-specific carts - Email notifications

Solution:

```
// Fast startup with lazy payment
@Component
@Primary
@Lazy
public class CreditCardPayment implements PaymentService { }

// User-specific state
@Component
@Scope("prototype")
```

```

public class ShoppingCart { }

// Default notification
@Component
@Primary
public class EmailNotification implements NotificationService { }

// Resource management
@Component
public class DatabaseService {
    @PostConstruct
    public void init() { /* open connection */ }

    @PreDestroy
    public void cleanup() { /* close connection */ }
}

```

Scenario 2: Banking Application

Requirements: - Multiple validators - Audit logging - Transaction isolation - Secure connections

Solution:

```

// Default validator
@Component
@Primary
public class CreditScoreValidator implements Validator { }

// Specific validator
@Component
public class IncomeValidator implements Validator { }

// Lazy audit
@Component
@Lazy
public class AuditService {
    @PostConstruct
    public void init() { /* connect to audit server */ }
}

// Per-transaction state
@Component
@Scope("prototype")
public class Transaction { }

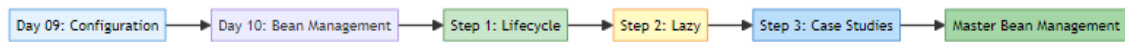
```

LEARNING PATH



📌 **Structured Learning by:** Avinash Dhanuka | © 2026

🎓 Recommended Learning Sequence



📖 Step-by-Step Guide

Step 1: Master Bean Lifecycle (2-3 hours)

Prerequisites: - Day 09: Annotation-based configuration - Understanding of @Component and @Autowired

Learning Path: 1. Read [BeanLifecycle/README.md](#) 2. Understand the 13 lifecycle phases 3. Run BeanLifecycle project 4. Experiment with @PostConstruct and @PreDestroy 5. Try different lifecycle methods

Hands-On Exercise:

```
// Create a FileService with proper Lifecycle
@Component
public class FileService {
    private FileWriter writer;

    @PostConstruct
    public void init() throws IOException {
        writer = new FileWriter("app.log");
        System.out.println("File opened");
    }

    public void log(String message) throws IOException {
        writer.write(message + "\n");
    }
}
```

```

    }

    @PreDestroy
    public void cleanup() throws IOException {
        writer.close();
        System.out.println("File closed");
    }
}

```

Key Takeaways: - @PostConstruct runs after DI - @PreDestroy runs before destruction - Proper resource management prevents leaks

Step 2: Understand Lazy Initialization (2-3 hours)

Prerequisites: - Step 1 completed - Understanding of bean creation

Learning Path: 1. Read [LazyConfig/README.md](#) 2. Understand eager vs lazy initialization 3. Run LazyConfig project 4. Experiment with @Lazy at different levels 5. Observe proxy behavior

Hands-On Exercise:

```

// Create expensive service with lazy loading
@Component
@Lazy
public class ReportService {
    public ReportService() {
        System.out.println("Loading report templates...");
        // Simulate expensive initialization
        try { Thread.sleep(3000); } catch (Exception e) {}
        System.out.println("ReportService ready");
    }

    public void generate() {
        System.out.println("Generating report...");
    }
}

// Test startup time difference
public static void main(String[] args) {
    long start = System.currentTimeMillis();
    ApplicationContext context = new
        AnnotationConfigApplicationContext(Config.class);
    long end = System.currentTimeMillis();
}

```

```

        System.out.println("Startup time: " + (end - start) + "ms");
    }

```

Key Takeaways: - @Lazy improves startup time - Proxy created for injection - First access is slower

Step 3: Apply in Real-World Scenarios (3-4 hours)

Prerequisites: - Steps 1 and 2 completed - Understanding of @Primary and @Qualifier

Learning Path: 1. Read [CaseStudy/README.md](#) 2. Study BankLoanApproval project 3. Study FoodDelivery project 4. Study PaymentProcessing project 5. Understand how concepts integrate

Hands-On Exercise:

```

// Build your own notification system
public interface NotificationService {
    void send(String message);
}

@Component
@Primary
public class EmailService implements NotificationService {
    @PostConstruct
    public void init() {
        System.out.println("Email service initialized");
    }

    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

@Component
@Lazy
public class SmsService implements NotificationService {
    public SmsService() {
        System.out.println("SMS gateway connected");
    }

    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}

```

```

    }
}

@Component
public class NotificationManager {
    private final NotificationService defaultService;
    private final NotificationService urgentService;

    public NotificationManager(
        NotificationService defaultService,
        @Qualifier("smsService") NotificationService urgentService) {
        this.defaultService = defaultService;
        this.urgentService = urgentService;
    }

    public void sendNormal(String msg) {
        defaultService.send(msg);
    }

    public void sendUrgent(String msg) {
        urgentService.send(msg);
    }
}

```

Key Takeaways: - Combine multiple concepts - @Primary for defaults - @Lazy for expensive services - Lifecycle for resource management

Practice Projects

Beginner Level

1. **Logger Service** - Lifecycle management for file logging
2. **Cache Manager** - Lazy initialization with @PostConstruct
3. **Config Loader** - Load properties in @PostConstruct

Intermediate Level

1. **Database Pool** - Lifecycle + Singleton scope
2. **Report Generator** - Lazy + expensive initialization
3. **Multi-Channel Notifier** - @Primary + @Qualifier

Advanced Level

1. **Payment Gateway** - All concepts combined
 2. **Order Processing** - Multiple scopes and lifecycle
 3. **Microservice Client** - Lazy + Lifecycle + Scopes
-

QUICK REFERENCE



Annotation Cheat Sheet

Annotation	Purpose	Example	When to Use
@PostConstruct	Initialization after DI	<code>@PostConstruct void init()</code>	Open connections, load config
@PreDestroy	Cleanup before destruction	<code>@PreDestroy void cleanup()</code>	Close connections, save state
@Lazy	Delay bean creation	<code>@Component @Lazy</code>	Expensive, rarely-used beans
@Scope("singleton")	One instance per container	<code>@Scope("singleton")</code>	Stateless services (default)
@Scope("prototype")	New instance per request	<code>@Scope("prototype")</code>	Stateful objects
@Primary	Default bean selection	<code>@Primary @Component</code>	Most common implementation
@Qualifier	Specific	<code>@Qualifier("beanName")</code>	Override @Pri-

Annotation	Purpose	Example	When to Use
	bean selection		many

🔍 Common Patterns

Pattern 1: Database Service

```

@Component
public class DatabaseService {
    private Connection connection;

    @PostConstruct
    public void init() {
        connection = DriverManager.getConnection(url);
    }

    @PreDestroy
    public void cleanup() {
        connection.close();
    }
}

```

Pattern 2: Lazy Expensive Service

```

@Component
@Lazy
public class ReportGenerator {
    public ReportGenerator() {
        // Expensive initialization
    }
}

```

Pattern 3: Multiple Implementations

```

@Component
@Primary
public class EmailNotification implements NotificationService { }

@Component
public class SmsNotification implements NotificationService { }

@Component

```



```

public class NotificationManager {
    public NotificationManager(
        NotificationService defaultService, // Gets @Primary
        @Qualifier("smsNotification") NotificationService smsService) {
    }
}

```

Pattern 4: Prototype with State

```

@Component
@Scope("prototype")
public class ShoppingCart {
    private List<Item> items = new ArrayList<>();
}

```

⚠ Common Mistakes

Mistake	Problem	Solution
Using constructor for DI logic	Dependencies not injected	Use @PostConstruct
Forgetting @PreDestroy	Resource leaks	Always cleanup resources
@PreDestroy on Prototype	Not called by Spring	Manual cleanup
Lazy everything	Late error detection	Lazy only expensive beans
Wrong scope	State pollution	Singleton for stateless, Prototype for stateful

📊 Performance Guidelines

Scenario	Recommendation	Reason
Startup > 10s	Use @Lazy for expensive beans	Improve user experience
Memory > 500MB	Review bean scopes	Reduce memory footprint
Rarely used feature	@Lazy	Save resources
Always used service	Eager (default)	Fail-fast, pre-

Scenario	Recommendation	Reason
		dictable
Stateful object	Prototype	Prevent state pollution
Stateless service	Singleton	Memory efficient

RUNNING THE PROJECTS



Prerequisites

- **Java 21** or higher
- **Maven 3.6+**
- **IDE** (IntelliJ IDEA, Eclipse, or VS Code)
- **Spring Framework 7.0.3**

Project 1: BeanLifecycle

Navigate to project:

```
cd day10/BeanLifecycle
```

Build:

```
mvn clean install
```

Run:

```
mvn exec:java -Dexec.mainClass="org.example.lifecycle.LifecycleDemo"
```

Expected Output:

```
--- Container Starting ---  
DB Constructor is called !!
```

```
Init Method is called !!
```

```
-- Using Bean --
```

```
Operation Successfully : Query is being Executed !!
```

```
SELECT * FROM students
```

```
--- Container Closing ---
```

```
Destroy method called : before Object Destruction
```

Project 2: LazyConfig

Navigate to project:

```
cd day10/LazyConfig
```

Build:

```
mvn clean install
```

Run:

```
mvn exec:java -Dexec.mainClass="org.example.lazy.LazyDemo"
```

Expected Output:

```
== Container Created ==
```

```
Eager Bean Created !!
```

```
== Requesting Lazy Bean ==
```

```
Lazy Bean Created !!
```

```
== Container Closed ==
```

Project 3: CaseStudy

Navigate to project:

```
cd day10/CaseStudy
```

Build:

```
mvn clean install
```

Run Bank Loan Approval:

```
mvn exec:java -Dexec.mainClass="org.example.bankloan.BankLoanDemo"
```

Run Food Delivery:

```
mvn exec:java -Dexec.mainClass="org.example.fooddelivery.FoodDeliveryDemo"
```

Run Payment Processing:

```
mvn exec:java -Dexec.mainClass="org.example.payment.PaymentDemo"
```

IDE Setup

IntelliJ IDEA: 1. File → Open → Select day10 folder 2. Wait for Maven import 3. Right-click on Demo class → Run

Eclipse: 1. File → Import → Existing Maven Projects 2. Select day10 folder 3. Right-click on Demo class → Run As → Java Application

VS Code: 1. Open day10 folder 2. Install Java Extension Pack 3. Click Run button in Demo class

Troubleshooting

Issue: Maven dependencies not downloading

```
mvn clean install -U
```

Issue: Java version mismatch

```
java -version # Should be 21+  
mvn -version # Should use Java 21+
```

Issue: Class not found

```
mvn clean compile
```

WHAT I LEARNED



📌 **Personal Learning Journey by:** Avinash Dhanuka | © 2026

🎓 Key Takeaways from Day 10

- 1. Bean Lifecycle is Critical for Resource Management**
 - @PostConstruct ensures dependencies are injected before initialization
 - @PreDestroy prevents resource leaks and memory issues
 - Proper lifecycle management is essential for production applications
 - Constructor is NOT the place for DI-dependent logic
- 2. Lazy Initialization Dramatically Improves Startup**
 - @Lazy can reduce startup time by 50-80%
 - Perfect for expensive, rarely-used beans
 - Creates proxy for dependency injection
 - Trade-off: first access is slower, but worth it
- 3. Bean Scopes Control Instance Creation**
 - Singleton (default) for stateless services
 - Prototype for stateful objects
 - Scope affects performance and memory
 - @PreDestroy doesn't work with Prototype
- 4. Dependency Resolution Provides Flexibility**
 - @Primary marks the default implementation
 - @Qualifier overrides @Primary for specific needs
 - Resolution priority: @Qualifier > Name > @Primary > Single
 - Only ONE @Primary per type allowed
- 5. Combining Concepts Solves Real Problems**
 - @Primary + @Lazy for default but expensive beans
 - Prototype + Lifecycle for per-request resources
 - @Qualifier + @Lazy for optional specific implementations
 - Real-world apps use multiple patterns together
- 6. Performance Optimization Requires Trade-offs**
 - Lazy loading: faster startup vs slower first access
 - Singleton: memory efficient vs potential state issues

- Prototype: state isolation vs memory overhead
 - Choose based on actual requirements
7. **Resource Management Prevents Production Issues**
- Always close database connections in @PreDestroy
 - Always release file handles and network sockets
 - Always flush buffers before shutdown
 - Memory leaks are silent killers in production
8. **Annotation Order and Placement Matter**
- @Lazy on class vs parameter has different effects
 - @Primary + @Lazy work together perfectly
 - @Scope affects lifecycle behavior
 - Understanding interactions is crucial

💡 Real-World Applications

E-Commerce Platform: - Lazy payment gateways (expensive connections) - Prototype shopping carts (per-user state) - Singleton product catalog (shared data) - Lifecycle for database pool management

Banking System: - @Primary for default validators - @Qualifier for specific validation rules - Prototype for transaction objects - Lifecycle for audit logging

Notification System: - @Primary for email (most reliable) - @Lazy for SMS (expensive gateway) - Singleton for notification manager - Lifecycle for connection management

CONCLUSION

📌 **Final Thoughts by:** Avinash Dhanuka | © 2026

Congratulations on completing Day 10! You've mastered advanced Spring bean management concepts that are essential for building production-ready applications.

What You've Achieved: - ✔ Mastered bean lifecycle management - ✔ Optimized performance with lazy loading - ✔ Applied concepts in real-world scenarios - ✔ Understood resource management patterns - ✔ Ready for Spring Boot and advanced topics

Remember: - @PostConstruct for initialization - @PreDestroy for cleanup -
@Lazy for performance - @Primary for defaults - @Qualifier for specific needs



Made with ❤ by Avinash Dhanuka



© 2026 Avinash Dhanuka. All Rights Reserved.

Keep coding, Keep learning! 🚀
